

**Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science
and Commerce**

Department of Information Technology (B.Sc.I.T Semester V)

AI

Practical – VI

Roll No.: T010	Name: Sagar Kandari
Class: TYIT	Batch: 01
Date of Assignment: 19/8/26	Date/Time of Submission: 19/8/26

Probability Reasoning (Bayesian Networks & Inference)

A. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code:-

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
```

```
)  
cpd_alarm = TabularCPD(  
    variable="Alarm",  
    variable_card=2,  
    values=[  
        [0.999, 0.71, 0.06, 0.05],  
        [0.001, 0.29, 0.94, 0.95]  
    ],  
    evidence=["Burglary", "Earthquake"],  
    evidence_card=[2, 2],  
    state_names={  
        "Alarm": ["False", "True"],  
        "Burglary": ["False", "True"],  
        "Earthquake": ["False", "True"]  
    }  
)  
cpd_john = TabularCPD(  
    variable="JohnCalls",  
    variable_card=2,  
    values=[  
        [0.95, 0.10],  
        [0.05, 0.90]  
    ],  
    evidence=["Alarm"],  
    evidence_card=[2],  
    state_names={  
        "JohnCalls": ["False", "True"],  
        "Alarm": ["False", "True"]  
    }  
)  
cpd_mary = TabularCPD(  
    variable="MaryCalls",  
    variable_card=2,  
    values=[  
        [0.99, 0.30],  
        [0.01, 0.70]  
    ],  
    evidence=["Alarm"],  
    evidence_card=[2],  
    state_names={
```

```
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)
assert model.check_model(), "Model check failed!"
infer = VariableElimination(model)
query_result = infer.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)
print(query_result)
```

Output:-

```
+-----+-----+
| Burglary | phi(Burglary) |
+=====+=====+
| Burglary(False) | 0.7158 |
+-----+-----+
| Burglary(True) | 0.2842 |
+-----+-----+
```

B. Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

Code:-

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
```

```
        ("Earthquake", "Alarm")
    ])

cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)

cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)

cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05],
        [0.001, 0.29, 0.94, 0.95]
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)

model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm
)

print("Is Bayesian Network valid?", model.check_model())
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)
```

```
)  
print("\nPosterior Probability:")  
print("P(Burglary | Alarm = True)")  
print(result)
```

Output:-

Is Bayesian Network valid? True

Posterior Probability:

P(Burglary | Alarm = True)

Burglary	phi(Burglary)
Burglary(False)	0.6264
Burglary(True)	0.3736

Extra Task (6A & 6B).

6A - Create a Bayesian Network for a real-world problem involving at least four variables using pgmpy.

Variables:

- Disease
- Fever
- Cough
- Fatigue

Network: Disease → Fever Disease → Cough Disease → Fatigue

Code:-

```
from pgmpy.models import DiscreteBayesianNetwork  
  
from pgmpy.factors.discrete import TabularCPD  
  
from pgmpy.inference import VariableElimination  
  
# Create Bayesian Network  
  
model = DiscreteBayesianNetwork([
```

```
( 'Disease', 'Fever'),  
  
    ( 'Disease', 'Cough'),  
  
    ( 'Disease', 'Fatigue')  
])  
  
# CPD for Disease  
  
cpd_disease = TabularCPD(  
  
    variable='Disease',  
  
    variable_card=2,  
  
    values=[[0.7],  
  
            [0.3]],  
  
    state_names={'Disease': ['No', 'Yes']}  
)  
  
# CPD for Fever  
  
cpd_fever = TabularCPD(  
  
    variable='Fever',  
  
    variable_card=2,  
  
    values=[  
  
        [0.8, 0.2],  
  
        [0.2, 0.8]  
  
    ],  
  
    evidence=['Disease'],  
  
    evidence_card=[2],  
  
    state_names={
```

```
        'Fever': ['No', 'Yes'],

        'Disease': ['No', 'Yes']

    }

)

# CPD for Cough

cpd_cough = TabularCPD(

    variable='Cough',

    variable_card=2,

    values=[

        [0.9, 0.3],

        [0.1, 0.7]

    ],

    evidence=['Disease'],

    evidence_card=[2],

    state_names={

        'Cough': ['No', 'Yes'],

        'Disease': ['No', 'Yes']

    }

)

# CPD for Fatigue

cpd_fatigue = TabularCPD(

    variable='Fatigue',

    variable_card=2,
```

Sagar T010

```
values=[

    [0.85, 0.4],

    [0.15, 0.6]

],

evidence=['Disease'],

evidence_card=[2],

state_names={

    'Fatigue': ['No', 'Yes'],

    'Disease': ['No', 'Yes']

}

)

# Add CPDs to the model

model.add_cpds(

    cpd_disease,

    cpd_fever,

    cpd_cough,

    cpd_fatigue

)

# Check the model

print("Model is valid:", model.check_model())

print("\nBayesian Network:")

print(model.edges())
```

6B - Perform inference to calculate a posterior probability for a given observation.

Code:-

```
# Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=['Disease'],
    evidence={
        'Fever': 'Yes',
        'Cough': 'Yes'
    }
)
print("\nPosterior Probability:")
print(result)
```

Output:-

```
Posterior Probability:
+-----+-----+
| Disease      | phi(Disease) |
+=====+=====+
| Disease(No)  | 0.0769       |
+-----+-----+
| Disease(Yes) | 0.9231       |
+-----+-----+
```